

Project Overview

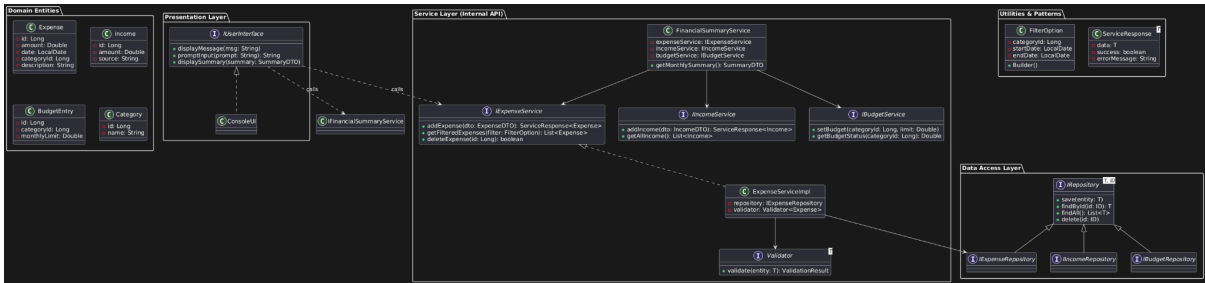
The objective is to build a console-based application to manage expenses, income, and budgets with persistent database storage. The design prioritizes **separation of concerns** to ensure the core logic remains independent of the interface.

System Architecture

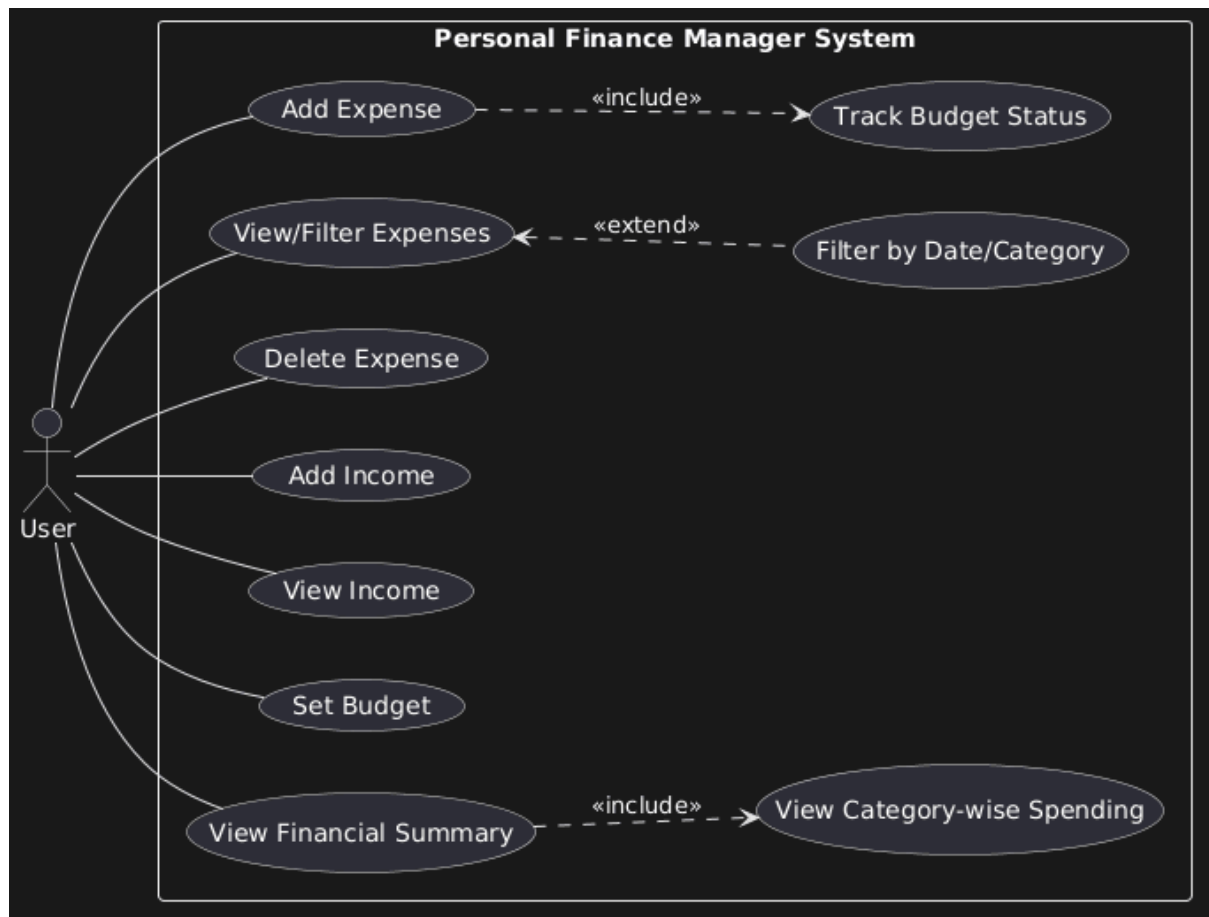
The system follows a **Layered Architecture** (inspired by Ports and Adapters) to ensure low coupling and high maintainability.

- **Presentation Layer (ConsoleUI):** Handles user interaction, input validation, and data display.
- **Service Layer (Business Logic):** Contains the "Internal API" where business rules—such as budget tracking and financial calculations—reside.
- **Data Access Layer (Repository):** Abstracts database operations, allowing the system to switch databases without affecting business logic.
- **Entity Layer:** Represents the core data objects (POJOs) like Expense, Income, and BudgetEntry.

Class Diagram



Use Case Diagram



Functional Components

Expense & Income Management

- **Functionality:** Users can add, view, and delete entries.
- **Filtering:** A `FilterOption` builder allows users to query data by specific dates or categories.

Budget Management

- **Tracking:** The system links expenses to specific categories to track spending against set limits.
- **Validation:** Every new expense is checked against the budget via a `Validator` interface to ensure data integrity.

Financial Summary

- **Orchestration:** The `FinancialSummaryService` aggregates data from both income and expense services to calculate total balance and category-wise spending.

Database Design (Relational)

A. Categories Table

This acts as a reference table for both Expenses and Budgets.

- **Columns:**
 - id (Primary Key, Autoincrement)
 - name (VARCHAR, Unique, Not Null) — e.g., "Transport", "Groceries".

B. Expenses Table

Stores individual spending records linked to a category.

- **Columns:**
 - id (Primary Key, Autoincrement)
 - amount (DECIMAL(10,2), Not Null) — Use Decimal for currency to avoid floating-point errors.
 - category_id (Foreign Key referencing Categories.id).
 - expense_date (DATE, Not Null).
 - description (VARCHAR) — Optional note for the user.

C. Incomes Table

Tracks money coming in. Unlike expenses, these may not need a strict category link but could have a "Source" field.

- **Columns:**
 - id (Primary Key, Autoincrement)
 - amount (DECIMAL(10,2), Not Null).
 - source (VARCHAR) — e.g., "Salary", "Freelance".
 - income_date (DATE, Not Null).

D. Budgets Table

Defines the spending limit for a specific category for a given timeframe.

- **Columns:**
 - id (Primary Key, Autoincrement)
 - category_id (Foreign Key referencing Categories.id).
 - monthly_limit (DECIMAL(10,2), Not Null).
 - month_year (VARCHAR/DATE) — To track budgets per month (e.g., "2026-05").

Design Patterns & Clean Code Principles

To meet the "Design Expectations" of the assignment, the following patterns are implemented:

- **Generic Repository Pattern:** Used to provide a standardized way to access data for all entities, reducing code duplication.
- **Dependency Inversion:** Services depend on interfaces (IExpenseRepository) rather than concrete classes, making the system easily testable.
- **Strategy Pattern (Validation):** Different validation rules can be swapped in or out for different entity types.
- **ServiceResponse Wrapper:** All service methods return a consistent response object containing data or error messages, facilitating better error handling

Error Handling & Validation

- **Input Validation:** The ConsoleUI ensures users enter valid data types (e.g., numbers for amounts, valid dates).
- **Domain Validation:** The Service layer checks for logical errors, such as negative spending or exceeding critical budget thresholds.
- **Persistence Safety:** Database connections are managed via a centralized manager to handle failures gracefully.